APOSTILA DE COMPILADORES

Aula 15

ANÁLISE SEMÂNTICA

Fechamento completo do front-end semântico do MINIC

Resolução de nomes • sistemas de tipos • compatibilidade e conversões • expressões • vetores • chamadas • retornos • fluxo de controle • diagnósticos

Ciência da Computação — 2026/2

Professor: Alex Torquato Souza Carneiro

Projeto integrador: compilador da linguagem MINIC

Apresentação e escopo desta aula

Esta versão revisada encerra a análise semântica prevista para o compilador MINIC. Os temas são tratados como uma unidade didática completa, encerrando os conteúdos que antecedem o início da representação intermediária.

A análise semântica recebe uma AST sintaticamente válida, resolve cada identificador no ambiente correto, determina tipos e verifica se declarações, expressões, chamadas e comandos respeitam as regras da linguagem. O resultado é uma AST validada e anotada — ou uma lista de diagnósticos —, ainda sem instruções intermediárias.

> **Fronteira explícita:** esta apostila não ensina código de três endereços, temporários, rótulos, quádruplas, triplas nem tradução de AST. Esses assuntos pertencem à representação intermediária, que começa na Aula 16 conforme a sequência didática solicitada.

Objetivos de aprendizagem

- Diferenciar erros sintáticos, erros semânticos estáticos e erros que só podem ser detectados em execução.
- Completar resolução de nomes, verificação de declarações e associação entre usos e símbolos.
- Aplicar regras explícitas de tipos primitivos, compostos, compatibilidade e conversões.
- Verificar operadores, atribuições, vetores, condições, chamadas, parâmetros e retornos.
- Validar o contexto de break e continue e a consistência de retorno das funções.
- Projetar diagnósticos úteis, evitar cascatas e testar sistematicamente as regras.

Como interpretar a especificação didática

O plano e as apostilas definem os tópicos, mas não fixam todos os detalhes da sintaxe concreta nem cada política de tipo da implementação da turma. Por isso, as regras abaixo constituem um contrato didático coerente para o núcleo MINIC. Quando a gramática ou a especificação oficial do projeto escolher outra grafia ou regra, ela prevalece; a equipe deve documentar e testar a decisão, sem deixar comportamento implícito.

1. O lugar da análise semântica

```
código-fonte → scanner → tokens → parser → AST
                                           ↓
                    tabela de símbolos e análise semântica
                                           ↓
                AST validada e anotada + diagnósticos
                                           ↓
                    representação intermediária
```

O scanner reconhece lexemas e posições; o parser verifica a estrutura gramatical e constrói a AST; a tabela de símbolos registra declarações e escopos. A análise semântica usa esses resultados para responder perguntas que a gramática, sozinha, não resolve: “este nome foi declarado?”, “este símbolo é uma função?”, “os operandos são compatíveis?” e “este return corresponde à função atual?”.

Camada	Exemplo	Responsabilidade
Sintaxe	f(, x);	A forma não respeita a gramática; responsabilidade do parser.
Semântica estática	int n; n = true;	A forma é válida, mas os tipos são incompatíveis.
Execução	v[i] com i inteiro fora da faixa	O tipo do índice é válido; o valor excede os limites em execução, salvo prova estática.

Estática e dinâmica

Uma verificação estática ocorre antes da execução. O MINIC deste projeto deve rejeitar estaticamente as violações cobertas nesta apostila. Verificações dinâmicas ocorrem quando valores concretos são conhecidos, como divisão por zero ou índice variável fora dos limites. Não se deve classificar como erro de tipo aquilo que depende de um valor calculado durante a execução.

“Estático” não significa que toda propriedade do programa pode ser decidida. Significa que uma regra definida para compilação é verificada sem executar o programa. Uma implementação pode complementar a verificação estática com proteções em tempo de execução, mas isso não substitui as regras de compilação aqui descritas.

2. Resolução de nomes e declarações

A resolução de nomes, introduzida na Aula 14, é pré-requisito para a verificação de tipos e fica encerrada como parte do contrato semântico. Cada uso de identificador deve ser ligado à declaração visível segundo escopo léxico; uma busca malsucedida é erro semântico, não erro de tipo.

Regras completas de declaração e busca

1. Ao declarar no escopo atual, rejeitar uma segunda declaração incompatível ou duplicada do mesmo nome naquele escopo; registrar a posição da primeira ocorrência.
2. Permitir sombreamento em escopo interno se a política MINIC adotada o permitir; a busca escolhe sempre a declaração visível mais interna.
3. Resolver cada uso consultando o escopo atual e depois seus pais. Se nenhuma declaração for encontrada, emitir “identificador não declarado”.
4. Verificar categoria: variável ou parâmetro pode ser lido; apenas entidade atribuível pode ser destino; função só pode ser chamada como função; vetor só pode ser indexado como vetor.
5. Registrar na AST a referência ao símbolo encontrado e manter seus atributos: categoria, tipo, assinatura ou dimensões e posição de declaração.

Declarações adiantadas, protótipos e recursão

A possibilidade de chamar uma função antes de sua definição depende da gramática e da política de declaração do MINIC. Para dar suporte a recursão e chamadas entre funções sem depender acidentalmente da ordem de visita, uma estratégia didática robusta é coletar primeiro as assinaturas globais e depois verificar os corpos. Se o projeto exigir declaração antes do uso,

a ordem deve ser aplicada uniformemente e documentada. Declarações duplicadas e conflitos entre protótipo e definição precisam ser detectados em qualquer política.

```
Passagem A — coletar declarações e assinaturas visíveis
Passagem B — criar ambientes de parâmetros e variáveis locais
Passagem C — resolver usos e verificar tipos/contextos
Resultado — símbolos associados, tipos anotados, erros acumulados
```

Observação: a estratégia de duas passagens é uma organização da análise semântica, não uma geração de código nem uma representação intermediária.

3. Sistema de tipos do MINIC

Um sistema de tipos associa um tipo às expressões e determina quais operações são válidas. O verificador pode sintetizar o tipo de cada nó a partir dos tipos dos filhos e dos símbolos resolvidos. Tipos tornam explícitos os valores que uma expressão pode produzir e os contextos nos quais esse valor pode ser usado.

Tipo	Uso no MINIC	Restrições essenciais
int (inteiro)	contagens, aritmética e índices	Operandos inteiros ou conversão numérica autorizada.
float (real)	valores fracionários	Operações numéricas; estreitamento não implícito.
bool (lógico)	verdadeiro/falso, condições	Operandos de &&, ||, ! e condições.
void	função sem valor de retorno	Não pode ser variável comum nem valor de expressão.
array(n,T)	vetor de n elementos do tipo T	Índice int; acesso produz T.
função $(T_1,\ldots,T_n)\rightarrow R$	assinatura de função	Aridade e tipos dos argumentos devem corresponder; retorno R.

As apostilas anteriores usam também os nomes “inteiro”, “real” e “lógico”. Eles são aqui sinônimos conceituais de int, float e bool; a implementação deve usar os nomes efetivamente definidos pelos tokens e pela gramática. void indica ausência de valor e é apropriado como tipo de retorno, não como tipo de objeto ordinário.

Síntese de tipo versus inferência

Na síntese, o tipo de um nó é calculado a partir dos tipos de seus filhos e da tabela de símbolos — abordagem adequada ao MINIC, cujos identificadores são declarados. Inferência geral, variáveis de tipo, unificação e polimorfismo são técnicas de outras linguagens e não são necessárias para o núcleo especificado neste curso. Não confundir síntese de tipos com adivinhação do tipo de uma variável não declarada.

4. Equivalência, compatibilidade e conversões

Equivalência estrutural

Dois tipos são estruturalmente equivalentes quando seus componentes têm a mesma forma e são equivalentes. Para vetores, a política MINIC adotada considera o tamanho e o tipo de elemento: array(4,int) equivale estruturalmente a array(4,int), mas não a array(5,int) nem a array(4,float). Para assinaturas funcionais, comparar número, ordem e tipo dos parâmetros e tipo do resultado.

Equivalência nominal

Na equivalência nominal, a identidade depende do nome declarado. Dois tipos nomeados de mesma estrutura continuam distintos se tiverem nomes diferentes. O MINIC básico descrito nas apostilas não introduz tipos agregados definidos pelo usuário (como struct ou typedef); portanto, equivalência nominal deve ser entendida como contraste teórico do plano, não como recurso a inventar na linguagem. Para os tipos efetivamente presentes, a decisão é estrutural/exata.

Compatibilidade não é equivalência

Compatibilidade é uma relação de uso: um valor de tipo S pode ou não ser aceito em um contexto que espera T. Um int pode ser compatível com um parâmetro float por promoção, sem que int e float sejam equivalentes. A regra precisa ser centralizada — por exemplo, compatível(valor, esperado, contexto) — para atribuições, argumentos, inicializadores e retornos apresentarem o mesmo comportamento.

Conversões permitidas

Origem → destino	Política didática	Exemplo
int → float	Promoção implícita permitida em contexto numérico.	float x; x = 2;
float → int	Conversão implícita proibida: pode perder informação.	int n; n = 2.7; — erro.
int ↔ bool	Sem conversão implícita.	if (1) — erro; comparar n != 0.
bool ↔ float/int	Sem conversão implícita.	true + 1 — erro.
cast explícito	Só permitido se constar da gramática e especificação.	Não aceitar sintaxe de cast por suposição.

Ao misturar int e float em uma operação numérica válida, o tipo comum é float e o operando int deve receber anotação de conversão. A anotação é semântica (tipo e coerção); a instrução concreta que realizará a conversão pertence à etapa de representação intermediária ou geração de código.

5. Regras de tipos para expressões e operadores

A função conceitual tipo(expr) visita recursivamente a árvore. Literais têm tipo determinado pelo token; identificadores herdam o tipo do símbolo resolvido; operadores consultam regras por categoria e devolvem um tipo ou ERROR. A tabela abaixo é a matriz normativa desta apostila.

Construção	Requisitos	Tipo do resultado
Literal inteiro / real / lógico	Lexema reconhecido pelo scanner.	int / float / bool
+ unário, - unário	Operando int ou float.	Tipo numérico do operando.
+, -, *, /	Dois operandos numéricos.	int se ambos int; caso contrário float.
% (se existir na gramática)	Dois operandos int.	int; em outro caso, erro.
<, <=, >, >=	Operandos numéricos; promoção int→float quando mistos.	bool
==, !=	Tipos comparáveis e compatíveis; números podem ser promovidos.	bool
&&, ||	Ambos os operandos bool.	bool
!	Operando bool.	bool
Atribuição	Destino atribuível; valor compatível com tipo esperado.	Comando; não produz valor no núcleo MINIC.

Aritmética

```
4 + 2       → int
4 + 2.5     → float  (4 recebe promoção int → float)
- x         → tipo numérico de x
true * 2    → erro: bool não é numérico
```

A regra para divisão descreve o tipo estático, não decide sozinha se a divisão de inteiros trunca, produz real ou gera exceção. A semântica operacional da linguagem deve fixar esse comportamento; independentemente dele, o verificador aplica a regra de tipo definida para / e não mistura regras sem documentá-las.

Comparações e lógica

Operadores relacionais e de igualdade produzem bool; não produzem o tipo dos operandos. Comparações encadeadas como a < b < c não são implicitamente uma operação única: aceitar ou rejeitar depende da gramática, mas a regra de tipos não pode reinterpretá-las silenciosamente. && e || exigem bool. A linguagem deve definir avaliação em curto-circuito para a execução; o verificador estático verifica os tipos de ambos os operandos, mesmo que um possa não ser avaliado em execução.

Atribuibilidade e valores-L/valores-R

Uma expressão usada como valor (valor-R) fornece um dado; um destino de atribuição precisa designar um local gravável (valor-L). Um identificador variável e um elemento v[i] podem ser destinos se a linguagem assim os define. Literal, chamada de função ou expressão aritmética não são destinos. O parser pode aceitar a forma geral; a verificação semântica confirma categoria e atribuibilidade.

```
int n; float r; bool b;
n = 4;       // válido
r = n;       // válido com promoção
n = 4.5;     // erro: estreitamento implícito
4 = n;       // erro: literal não é destino
b = (n < 8); // válido se a gramática permitir atribuição de bool
```

6. Declarações, inicializadores e atribuições

Cada declaração deve ser verificada no escopo apropriado e criar uma entrada coerente na tabela de símbolos. Inicializadores — se permitidos pela gramática — obedecem às mesmas regras de compatibilidade da atribuição. Declarar duas vezes no mesmo escopo é erro; sombrear em um bloco interno segue a política de escopo da Aula 14.

Verificação	Regra	Diagnóstico típico
Duplicidade	Mesmo nome já declarado no escopo corrente.	“x já foi declarado neste escopo”; citar a declaração anterior.
Uso sem declaração	Nenhum símbolo visível com aquele nome.	“identificador x não declarado”.
Categoria	Uso como chamada, vetor, variável ou destino deve combinar com a categoria.	“f não designa uma função”.
Inicialização /atribuição	O destino é gravável e o valor é compatível.	“não se pode atribuir float a int sem conversão permitida”.

Não criar um diagnóstico secundário de tipo para um identificador que já falhou na resolução. Marcar seu tipo como ERROR permite que a análise continue sem afirmar falsamente que o erro original era “operador inválido”.

7. Vetores: declaração, acesso e limites

Um vetor tem tipo array(n,T): dimensão ou tamanho n e tipo de elemento T. A tabela de símbolos deve distinguir vetor de variável escalar e guardar as dimensões necessárias. Para v[i], verificar primeiro que v resolve para vetor, depois que i é int; o tipo do acesso é T. Se vetores multidimensionais fizerem parte da gramática, cada índice consome uma dimensão e o acesso parcial tem o tipo do subvetor restante.

```
float notas[4];
int i;
notas[i] = 8.5;   // válido; elemento é float
notas[1.5] = 8;   // erro estático: índice deve ser int
i[0] = 3;         // erro: i não é vetor
notas[0] = true;  // erro: valor bool incompatível com float
```

Tipo de índice e faixa são verificações distintas. Um índice float é erro estático. Um índice int variável cujo valor real seja negativo ou maior que o limite é erro de execução, a menos que o valor seja constante e o compilador prove a violação ou a validade. Caso o tamanho do vetor seja conhecido e o índice seja literal, pode-se detectar fora de faixa na compilação sem confundir isso com incompatibilidade de tipos.

Obrigatório para o verificador: tipo int para cada índice, categoria vetor no objeto indexado, resultado com tipo do elemento e validação do uso como destino.

8. Funções, assinaturas e chamadas

A entrada da tabela para uma função contém sua categoria, lista ordenada de tipos de parâmetros, tipo de retorno e localização da declaração/definição. A assinatura determina como validar toda chamada: resolver o nome, confirmar que é função, comparar aridade e então compatibilidade de cada argumento com o parâmetro correspondente.

1. Resolver o identificador da função no escopo apropriado; se não existir, reportar nome não declarado.
2. Se o nome existir mas representar variável ou vetor, reportar categoria incorreta.
3. Comparar o número de argumentos com o número de parâmetros. Se divergir, reportar aridade e evitar mensagens repetidas para posições inexistentes.
4. Para cada posição existente, verificar o tipo do argumento segundo o parâmetro correspondente, incluindo somente conversões permitidas.
5. Atribuir à expressão da chamada o tipo de retorno da assinatura; se for void, a chamada não fornece valor.

Caso	Resultado
int soma(int a, float b); soma(3, 2.5)	Válido; chamada tem tipo int.
soma(3)	Erro de aridade: esperados 2, recebido 1.
soma(true, 2.5)	Erro no argumento 1: bool incompatível com int.
void imprime(int x); imprime(2);	Válido como comando.
n = imprime(2);	Erro: chamada void usada como valor.

Se houver declarações de protótipo e definição, as assinaturas devem ser comparadas e a definição única exigida. A regra sobre funções chamadas antes da definição precisa ser a mesma adotada na seção de resolução de nomes. Recursão direta ou mútua requer que as assinaturas relevantes estejam disponíveis antes da verificação dos corpos, caso seja permitida pela especificação.

9. Retornos e tipo da função

Ao verificar o corpo de uma função, o analisador mantém o tipo de retorno esperado no contexto. Cada return é comparado com esse tipo. Return não é apenas uma expressão: envolve contexto de função, presença/ausência de valor e, para garantia de execução, cobertura dos caminhos de controle.

Função	Comando	Regra
int f()	return 3;	Válido.
int f()	return 2.5;	Erro: float não estreita implicitamente para int.
int f()	return;	Erro: função exige valor.
void g()	return;	Válido.
void g()	return 3;	Erro: função void não retorna valor.
float h()	return 2;	Válido com promoção int→float.

Retorno em todos os caminhos

Além de validar cada return individual, a análise deve verificar se uma função não void pode alcançar o fim do corpo sem retornar um valor. Uma regra conservadora considera garantido o retorno quando o bloco termina com return; quando ambos os ramos de um if/else garantem retorno; ou quando uma sequência anterior já torna o fim inalcançável. Um while comum não garante execução e, portanto, não prova por si só que a função retorna. Se a linguagem admitir funções não void sem retorno em todos os caminhos, essa política precisa estar explicitada; para o projeto didático, recomenda-se rejeitar a possibilidade de queda pelo fim.

```
int abs(int x) {
    if (x < 0) return -x;
    else return x;  // ambos os ramos retornam
}

int incompleta(bool b) {
    if (b) return 1;
    // caminho b == false alcança o fim: erro
}
```

10. Condições, break e continue

As condições de if e while devem ter tipo bool. Não se converte int em bool por “verdadeiro se não zero” a menos que a especificação MINIC diga isso expressamente; na política deste material, tal coerção não existe. Uma comparação, como x != 0, produz bool e pode ser usada como condição.

break e continue são erros de contexto, não de tipo: ambos só podem ocorrer dentro de um laço. O verificador mantém loopDepth: aumenta ao entrar no corpo de while e diminui ao sair. Se MINIC incorporar outro comando iterativo, esse comando também atualiza a profundidade.

return, break e continue dentro de blocos aninhados continuam válidos somente conforme o contexto envolvente.

```
if (x < 10) x = x + 1;  // válido: comparação é bool
if (x) x = 1;             // erro: int não é condição bool
while (pronto && i < n) {
    if (i == 3) continue; // válido dentro do laço
    if (i > 8) break;     // válido dentro do laço
}
break;                    // erro fora de laço
```

Análise de fluxo necessária para retornos

A checagem de retorno em todos os caminhos compartilha uma análise simples de fluxo com a validação de comandos. Ela não gera saltos nem código: classifica se um comando pode terminar normalmente, sempre retorna, ou transfere controle por break/continue. Essa informação é suficiente para a regra conservadora de queda pelo fim da função e para evitar tratar laços como garantia de retorno.

Comando	Pode terminar normalmente?	Garantia de retorno
Comando simples	Sim	Não.
return e; / return;	Não	Sim (retorno executado).
if (c) S1 else S2	Depende dos ramos	Sim se os dois ramos garantem retorno.
if (c) S1 sem else	Sim pelo ramo ausente	Não, mesmo que S1 retorne.
while (c) S	Sim, pode executar zero vezes	Não por si só.
Bloco S1; S2	Composição sequencial	Sim se a sequência garante retorno antes da queda.

11. Arquitetura de implementação semântica

A implementação pode organizar verificações em uma ou mais travessias da AST. Independentemente do desenho, todos os resultados semânticos precisam ser determinísticos e preservados: referência do identificador ao símbolo, tipo sintetizado, conversões permitidas e posição para diagnóstico. Os dados de entrada (AST, tabela e posições) não devem ser destruídos para produzir essas anotações.

Ordem recomendada de trabalho

1. Coletar declarações globais e detectar duplicidades/conflitos, conforme a política de ordem de declaração.
2. Abrir ambientes de função e bloco; inserir parâmetros e declarações locais; restaurar escopo ao sair.
3. Resolver usos e categorias antes de depender dos seus tipos.
4. Sintetizar tipos de literais, identificadores, operadores, acessos e chamadas.
5. Verificar contextos dos comandos: condição, laço, retorno e atribuibilidade.
6. Executar análise simples de fluxo para quedas sem retorno em funções não void.
7. Acumular diagnósticos, ordenar por posição de origem e devolver status semântico de sucesso/falha.

Anotações na AST

Nó	Atributos semânticos esperados
Identifier	símbolo resolvido, tipo, categoria, posição.
Literal	tipo primitivo e valor léxico convertido quando aplicável.
Binary / Unary	tipo dos operandos, conversões necessárias, tipo resultado.
Assign	tipo do destino e do valor, validade do destino e compatibilidade.
Index	tipo do vetor, tipo do índice e tipo do elemento.
Call	símbolo/assinatura, tipos dos argumentos, tipo de retorno.
Return	tipo esperado do contexto de função, tipo efetivo da expressão.
If / While / Break / Continue	tipo booleano da condição e contexto de laço.

As regras da Aula 13 sobre atributos sintetizados e herdados aplicam-se diretamente: tipos de expressões geralmente são sintetizados de baixo para cima; o tipo de retorno da função e a profundidade de laço são informações de contexto herdadas ao visitar comandos. O grafo de dependências deve ser acíclico para a ordem de avaliação escolhida.

12. Diagnósticos e recuperação semântica

Um diagnóstico deve ser acionável: apontar linha e coluna do uso problemático, identificar o nome ou operador, explicar a expectativa e, se útil, mostrar o tipo recebido e a posição da declaração conflitante. Usar categorias estáveis de erro facilita testes automatizados e integração com a etapa anterior.

Código/categoria	Exemplo	Mensagem sugerida
SEM001 não declarado	x = 1;	Identificador “x” não declarado neste escopo.
SEM002 duplicado	int x; float x;	“x” já declarado neste escopo (declaração anterior na linha …).
SEM003 incompatível	int n; n = 2.5;	Não é possível atribuir float a int sem conversão permitida.
SEM004 operador	true * 2	Operador * requer operandos numéricos; recebeu bool e int.
SEM005 condição	if (3)	Condição deve ter tipo bool; recebeu int.
SEM006 índice	v[1.2]	Índice do vetor deve ser int; recebeu float.
SEM007 chamada	f(1) com 2 parâmetros	“f” espera 2 argumentos, mas recebeu 1.
SEM008 argumento	f(true) com f(int)	Argumento 1 de “f”: esperado int, recebido bool.
SEM009 retorno	int f(){return true;}	Retorno bool incompatível com tipo int da função.
SEM010 contexto	break fora do laço	break só pode ocorrer dentro de um laço.

Código/categoria	Exemplo	Mensagem sugerida
SEM011 queda	função int sem retorno em um caminho	A função pode terminar sem retornar int.
SEM012 void	x = g(); com g void	Função sem valor de retorno usada em expressão.

Evitar mensagens em cascata

Use um tipo sentinela ERROR após erro primário. Regras que recebem ERROR devem propagar ERROR sem produzir uma mensagem derivada que não ajudaria o usuário. Ainda assim, continue examinando regiões independentes: um erro em um comando não deve ocultar outros erros de uma função. Defina um limite razoável de mensagens para evitar cascatas em entradas severamente inválidas.

13. Plano de testes e critérios de conclusão

A análise semântica só está concluída quando há testes positivos e negativos para cada classe de regra. Uma suíte pequena, organizada por categoria, distingue defeito do parser, da resolução de nomes e do verificador de tipos. Preservar a AST e comparar os diagnósticos esperados evita que uma regra nova esconda regressões anteriores.

Grupo de testes	Casos mínimos
Nomes e escopos	identificador resolvido; não declarado; duplicado no mesmo escopo; sombreado; saída de escopo restaura símbolo externo.
Atribuições	mesmo tipo; int para float; float para int; destino literal; categorias incompatíveis.
Operadores	todos os aritméticos válidos; bool em aritmética; relacionais; igualdade; &&, ||, !; mistura numérica.
Vetores	índice int; índice float; escalar indexado; tipo do elemento; atribuição a elemento; literal fora de faixa.
Funções	chamada válida; nome ausente; símbolo que não é função; aridade a menos/mais; erro em argumento específico; função void como valor.
Retornos	retorno compatível; incompatível; retorno ausente em função não void; valor em void; caminho faltante.
Controle	condição bool e não bool; break/continue dentro e fora de laço; laços aninhados.
Diagnósticos	linha/coluna; posição da declaração anterior; acumulação independente; supressão de cascata.

Programa integrado de aceitação

```
int soma(int a, int b) { return a + b; }
float promove(int n) { return n; }
int principal() {
    int i;
    float x;
    bool ok;
    i = soma(2, 3);
    x = promove(i) + 0.5;
    ok = x > 2.0;
    while (ok && i < 10) {
        i = i + 1;
        if (i == 7) continue;
        if (i > 8) break;
    }
    return i;
}
```

Esse exemplo é apenas ilustrativo: nomes de tipos, sintaxe de declaração, presença de principal e regras de declaração devem coincidir com a gramática MINIC do repositório da turma. O programa deve ser aceito se essas construções forem suportadas. Em testes negativos, altere uma regra por vez — por exemplo, ok = 1, x = true, soma(2), notas[1.5], return; em int — e verifique categoria e posição do diagnóstico.

Checklist de encerramento semântico

- ☐ Todos os identificadores usados foram resolvidos ou diagnosticados; duplicidades e categorias foram verificadas.
- ☐ Todos os nós de expressão recebem tipo ou ERROR; operadores seguem uma matriz única de regras.
- ☐ Compatibilidade, equivalência estrutural/nominal e conversões estão explicitadas e testadas.
- ☐ Atribuições e inicializadores validam destino atribuível e tipo do valor.
- ☐ Vetores validam categoria, tipo do índice, tipo do elemento e limites constantes detectáveis.
- ☐ Chamadas validam símbolo função, aridade, tipos e uso de retorno void.
- ☐ Cada return é compatível com a assinatura; funções não void não podem cair pelo fim segundo a política adotada.
- ☐ Condições exigem bool; break e continue exigem contexto de laço.
- ☐ Mensagens contêm posição, causa e tipos relevantes; erros em cascata são controlados.
- ☐ A AST permanece disponível e anotada, sem conter código de três endereços, rótulos ou temporários.

Critério de transição: quando todos os itens do checklist estiverem satisfeitos, não resta conteúdo semântico previsto no plano para uma aula posterior. A próxima etapa pode receber uma AST validada e iniciar a representação intermediária.

14. Atividades de consolidação

Atividade A — matriz de tipos

Para cada expressão abaixo, indique os tipos dos operandos, a conversão aplicada (se houver), o tipo do resultado e se há erro: 7 + 2.0; !true; 5 < 8.0; false && (3 > 1); 3.5 % 2 (se % existir); 1 + false. Justifique cada resposta pela matriz da seção 5.

Atividade B — chamada e retorno

Considere float media(int soma, int quantidade). Classifique media(10, 2), media(10), media(true, 2) e uma atribuição int resultado = media(10,2). Proponha diagnóstico para cada programa inválido e discuta divisão por zero como questão estática ou dinâmica.

Atividade C — contexto de controle

Percorra uma AST com dois while aninhados, um continue no laço interno e um break após o fechamento de ambos. Registre loopDepth em cada nó e identifique qual comando é válido. Em seguida, determine se uma função não void garante retorno nos exemplos if sem else, if-else com dois returns e while com return no corpo.

Atividade D — implementação

Implemente em C ou Python as funções conceituais resolver(nome, ambiente), tipo(expressao), verificarAtribuicao(destino, valor), verificarChamada(call), verificarRetorno(return, funcao) e verificarComando(no, contexto). Inclua linha e coluna em todo nó e use ERROR para supressão de cascata.

Síntese

A análise semântica fecha a interpretação estática da AST: associa identificadores a declarações, confere duplicidades e categorias, sintetiza tipos, valida compatibilidade e conversões, analisa operações, atribuições, vetores, chamadas, argumentos, retornos e contextos de controle. As verificações dependentes de valores concretos permanecem em execução, salvo quando a compilação consegue prová-las.

A saída desta aula é uma AST semanticamente validada e anotada, acompanhada dos diagnósticos encontrados. A seguir, introduziremos a representação intermediária como próxima camada do compilador.

Referências

AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. Compiladores: princípios, técnicas e ferramentas. Livro-texto fornecido para a disciplina. Capítulo 6, especialmente as seções sobre tipos, verificação de tipos, conversões e fluxo de controle; Capítulo 2, seção sobre verificação estática e valores-L/valores-R.